

HAM Intelligence Guide

Overview

HAM (Hierarchical Access Memory) Intelligence extends the base HAM system with entropy-based scoring, automatic tier management, compression, and NUMA awareness. This creates a self-optimizing memory substrate that learns from access patterns.

Core Concepts

Memory Tiers

HAM_CRITICAL	☞	Hot, frequently accessed, pinned in fastest memory
HAM_ACTIVE	☞	Warm, general purpose, standard RAM
HAM_DORMANT	☞	Cool, infrequently accessed, compressible
HAM_ARCHIVE	☞	Cold, rarely accessed, offloadable to SSD

Entropy Scoring

Entropy measures the “randomness” of access patterns:

- **Low entropy** (< 1.0): Regular, predictable access → HOT data
- **High entropy** (> 2.0): Irregular, unpredictable access → COLD data

Entropy-Based Scoring

Shannon Entropy

```
float ham_compute_entropy(const void* data, size_t size);
```

Computes Shannon entropy of byte distribution:

$$H(X) = -\sum p(x) * \log_2(p(x))$$

Where:

- $p(x)$ = probability of byte value x
- Range: [0, 8] bits for byte data

Access Pattern Entropy

```
float access_pattern_compute_entropy(const AccessPattern* pattern);
```

Analyzes inter-access time deltas:

$$\Delta t = [t_1 - t_0, t_2 - t_1, t_3 - t_2, \dots]$$

Quantizes into buckets:

- Bucket 0: $\Delta t < 10$ cycles (very frequent)
- Bucket 1: $10 \leq \Delta t < 100$ (frequent)
- Bucket 2: $100 \leq \Delta t < 1000$ (occasional)
- Bucket 3: $\Delta t \geq 1000$ (rare)

API

```
#include "kernel/ham/entropy/ham_entropy.h"

// Compute data entropy
uint8_t data[1024] = /* ... */;
float entropy = ham_compute_entropy(data, 1024);
printf("Data entropy: %.2f bits\n", entropy);

// Track access pattern
AccessPattern* pattern = access_pattern_create(256);
access_pattern_record(pattern, current_cycle);
float access_entropy = access_pattern_compute_entropy(pattern);
```

Automatic Tier Management

Policy Configuration

```
typedef struct {
    float promotion_threshold; // Entropy threshold for promotion
    float demotion_threshold; // Entropy threshold for demotion
    uint64_t access_window; // Cycles to consider
    bool auto_compression; // Enable automatic compression
} HamPolicy;
```

Tier Manager

```
#include "kernel/ham/tier/ham_tier_manager.h"

// Create manager
HamPolicy policy = {
    .promotion_threshold = 0.5f, // Promote if entropy < 0.5
    .demotion_threshold = 2.0f, // Demote if entropy > 2.0
    .access_window = 1000, // Consider last 1000 cycles
    .auto_compression = true
};

HamTierManager* manager = ham_tier_manager_create(policy);

// Add regions
ham_tier_manager_add_region(manager, region1);
ham_tier_manager_add_region(manager, region2);

// Automatic tier management
ham_tier_manager_update(manager, current_cycle);
```

Promotion Logic

```
bool should_promote(const HamRegion* region,
                   const AccessPattern* pattern,
                   const HamPolicy* policy) {
    // Promote if:
    // 1. Low entropy (regular access)
    if (pattern->entropy_score < policy->promotion_threshold) {
        return true;
    }

    // 2. High access frequency
    if (region->stats.access_count > 1000) {
        return true;
    }

    return false;
}
```

Demotion Logic

```
bool should_demote(const HamRegion* region,
                  const AccessPattern* pattern,
                  const HamPolicy* policy) {
    // Demote if:
    // 1. High entropy (irregular access)
    if (pattern->entropy_score > policy->demotion_threshold) {
        return true;
    }

    // 2. Low access frequency
    if (region->stats.access_count < 10) {
        return true;
    }

    return false;
}
```

Compression via Motif Interning

Concept

Identifies repeated byte patterns (motifs) and replaces them with references.

```
Original: [AB AB AB CD AB AB EF AB AB]
Motifs:   [AB] appears 6 times
Compressed: [M0 M0 M0 CD M0 M0 EF M0 M0] + Dictionary[M0 = AB]
```

API

```
#include "kernel/ham/compression/ham_compression.h"

// Compress region
HamRegion* region = /* ... */;
if (ham_compress_region(region) == 0) {
    CompressionStats stats = ham_get_compression_stats(region);
    printf("Compression ratio: %.2fx\n", stats.compression_ratio);
    printf("Saved: %zu bytes\n",
           stats.original_size - stats.compressed_size);
}

// Decompress when needed
ham_decompress_region(region);
```

Motif Extraction

```
Motif** motif_extract(const void* data, size_t size, size_t* out_count);
```

Algorithm:

1. Scan data for repeated patterns (4-byte sequences)
2. Count frequency of each pattern
3. Create motif for patterns appearing 3+ times
4. Sort by frequency × pattern_length (compression benefit)

Compression Statistics

```
typedef struct {
    size_t original_size;
    size_t compressed_size;
    float compression_ratio;
    size_t num_motifs;
} CompressionStats;
```

NUMA Awareness

Concept

NUMA (Non-Uniform Memory Access) systems have multiple memory nodes with different access latencies. HAM Intelligence places memory regions on optimal NUMA nodes.

```
CPU 0 ↔ Memory Node 0 (local, fast)
↓
CPU 1 ↔ Memory Node 1 (remote, slow)
```

Affinity Computation

```
float ham_compute_numa_affinity(const HamRegion* region, uint32_t node_id);
```

Factors:

1. **Access frequency:** Higher frequency → higher affinity

2. **Memory tier**: CRITICAL tier → higher affinity
3. **Distance**: Closer nodes → higher affinity

NUMA Manager

```
#include "kernel/ham/numa/ham_numa.h"

// Create manager for 4 NUMA nodes
NumaManager* manager = numa_manager_create(4);

// Add regions
numa_manager_add_region(manager, region1);
numa_manager_add_region(manager, region2);

// Optimize placement
numa_manager_optimize(manager);
```

Migration Decision

```
bool should_migrate(const HamRegion* region,
                    uint32_t current_node,
                    uint32_t target_node) {
    float current_affinity = ham_compute_numa_affinity(region, current_node);
    float target_affinity = ham_compute_numa_affinity(region, target_node);

    // Require 20% improvement to justify migration cost
    return (target_affinity > current_affinity * 1.2f);
}
```

Integration Example

Complete HAM Intelligence Pipeline

```
#include "kernel/ham/entropy/ham_entropy.h"
#include "kernel/ham/tier/ham_tier_manager.h"
#include "kernel/ham/compression/ham_compression.h"
#include "kernel/ham/numa/ham_numa.h"

void optimize_memory_system(HamRegion** regions, size_t count) {
    // 1. Create tier manager
    HamPolicy policy = {0.5f, 2.0f, 1000, true};
    HamTierManager* tier_mgr = ham_tier_manager_create(policy);

    // 2. Create NUMA manager
    NumaManager* numa_mgr = numa_manager_create(4);

    // 3. Add all regions
    for (size_t i = 0; i < count; i++) {
        ham_tier_manager_add_region(tier_mgr, regions[i]);
        numa_manager_add_region(numa_mgr, regions[i]);
    }

    // 4. Main optimization loop
    for (uint64_t cycle = 0; cycle < 10000; cycle++) {
        // Update tier assignments
        ham_tier_manager_update(tier_mgr, cycle);

        // Compress dormant regions
        for (size_t i = 0; i < count; i++) {
            if (regions[i]->tier == HAM_DORMANT) {
                ham_compress_region(regions[i]);
            }
        }

        // Optimize NUMA placement every 1000 cycles
        if (cycle % 1000 == 0) {
            numa_manager_optimize(numa_mgr);
        }
    }

    // 5. Cleanup
    ham_tier_manager_free(tier_mgr);
    numa_manager_free(numa_mgr);
}
```

Performance Metrics

Entropy Computation

- **Time:** $O(n)$ for n bytes
- **Space:** $O(1)$ (256-element frequency array)
- **Typical:** ~ 1 GB/s throughput

Tier Management

- **Time:** $O(k)$ for k regions per update
- **Space:** $O(k)$ for access patterns

- **Typical:** $\sim 1 \mu\text{s}$ per region

Compression

- **Time:** $O(n^2)$ for motif extraction (can be optimized)
- **Space:** $O(m)$ for m motifs
- **Typical:** 2-5x compression for repetitive data

NUMA Optimization

- **Time:** $O(k \cdot n)$ for k regions, n NUMA nodes
- **Space:** $O(k + n)$
- **Typical:** $\sim 10 \mu\text{s}$ per optimization pass

Best Practices

1. Tune Policy Thresholds

```
// For ML workloads (regular access patterns)
HamPolicy ml_policy = {
    .promotion_threshold = 0.8f,    // Aggressive promotion
    .demotion_threshold = 1.5f,    // Conservative demotion
    .access_window = 10000,
    .auto_compression = false      // Disable for hot data
};

// For database workloads (mixed patterns)
HamPolicy db_policy = {
    .promotion_threshold = 0.5f,
    .demotion_threshold = 2.0f,
    .access_window = 1000,
    .auto_compression = true
};
```

2. Monitor Compression Ratios

```
void monitor_compression(HamRegion* region) {
    CompressionStats stats = ham_get_compression_stats(region);

    if (stats.compression_ratio < 1.2f) {
        // Poor compression, decompress
        ham_decompress_region(region);
    }
}
```

3. Balance NUMA Migration

```
// Avoid thrashing
static uint64_t last_migration[MAX_REGIONS] = {0};

bool can_migrate(RegionId id, uint64_t current_cycle) {
    const uint64_t MIN_INTERVAL = 10000; // 10k cycles
    return (current_cycle - last_migration[id]) > MIN_INTERVAL;
}
```

See Also

- [Graph Optimization Guide: Memory-efficient graph representations](#)
- [Device Backend API: Device-specific memory requirements](#)
- [Scheduler Guide: Memory-aware scheduling](#)